



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 7

Team No.: 23

Project Title: Asynchronous File Processing Server

Team Members

Roll Number	Student Name	Signature
2520030138	Hruday Deepak	
2520030135	Jaiswal Reddy	

1. Abstract

The Asynchronous File Processing Server is a Linux-based system programming project designed to efficiently handle a large number of concurrent file and network requests. In a traditional synchronous server, each request may block while performing file operations such as opening, reading, or writing, which can reduce server performance and scalability. This project proposes an asynchronous server implemented in C that processes multiple client requests without requiring a separate blocking thread for every operation. The server accepts concurrent client connections, places incoming requests into a request queue, and performs file I/O asynchronously using Linux asynchronous I/O mechanisms. A completion queue is used to track completed operations and return results to the appropriate clients. Synchronization mechanisms are applied to safely manage shared request and completion data. The server also records request and completion timestamps to calculate throughput and request latency. Relevant Operating Systems concepts include asynchronous I/O, process and thread concurrency, inter-process communication, synchronization, socket programming, queues, and system calls. The expected outcome is a lightweight and scalable file-processing server capable of handling multiple concurrent requests efficiently while providing performance measurements for throughput and latency.

2. Problem Statement

Cloud storage systems receive a large number of simultaneous file operations such as file opening, reading, writing, and data transfer. Processing these requests synchronously can cause blocking, increased waiting time, and reduced server throughput. Creating a separate blocking thread for every client request can also introduce significant resource overhead when the number of requests increases.

The project aims to develop an **asynchronous file-processing server in C on Linux** that can handle multiple client connections and file operations efficiently. The server will maintain request and completion queues, perform file I/O asynchronously, synchronize access to shared data, and measure request latency and server throughput. This project is

relevant to Operating Systems and Systems Programming because it demonstrates concurrency, asynchronous I/O, synchronization, socket communication, queues, system calls, and resource management.

3. Objectives

1. Develop a Linux-based asynchronous file-processing server using C.
2. Accept and manage multiple concurrent client connections.
3. Process file-read and file-write requests asynchronously.
4. Maintain a queue of pending I/O requests.
5. Maintain a completion queue for completed operations and their results.
6. Use synchronization mechanisms to safely access shared request data.
7. Avoid unnecessary blocking and improve server scalability.
8. Record request start and completion times.
9. Calculate request latency and overall server throughput.
10. Test server performance under multiple concurrent client requests.

4. Proposed Methodology

The server will be implemented in **C on Linux** using socket programming and Linux asynchronous I/O facilities.

1. **Server Initialization:**
The server creates a TCP socket, binds it to a specified port, and listens for incoming client connections.
2. **Client Connection Handling:**
Multiple clients can connect to the server concurrently. Incoming requests are received without creating a separate blocking thread for every request.
3. **Request Queue:**
Each client request contains information such as the operation type, file name, data, and request timestamp. Requests are placed into a thread-safe pending-request queue.
4. **Asynchronous File Processing:**
File read and write operations are submitted to the Linux asynchronous I/O mechanism, preferably **io_uring**, allowing I/O operations to proceed without continuously blocking the main server.
5. **Completion Queue:**
Once an asynchronous operation finishes, its completion information is obtained and placed into a completion queue. The corresponding result is then returned to the client.
6. **Synchronization:**
Mutexes, condition variables, or other synchronization mechanisms will protect shared queues and prevent race conditions when multiple execution units access request data.
7. **Performance Monitoring:**
The server records request arrival, processing, and completion times using Linux timing APIs. These measurements are used to calculate **request latency, completed requests per second, and overall throughput**.
8. **Testing and Evaluation:**
Multiple clients will be used to generate simultaneous read and write requests. The system will be evaluated based on concurrency, response time, throughput, and resource utilization.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Socket API (socket(), bind(), listen(), accept())	Establishes and manages multiple client-server network connections
Asynchronous I/O (io_uring)	Performs file read/write operations asynchronously without blocking the server.
POSIX Threads (pthread)	Provides concurrent processing for handling requests and background server tasks.
Mutex & Condition Variables (pthread_mutex, pthread_cond)	Synchronizes access to shared request and completion queues and prevents race conditions.
File System Calls (open(), read(), write(), close())	Performs file opening, reading, writing, and closing operations.
High-Resolution Timing (clock_gettime())	Records request and completion times to calculate latency and server throughput.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030135	Jaiswal Reddy	Server design, socket programming and client connection handling
2520030138	Hruday Deepak	Asynchronous file I/O, request queues, synchronization and performance testing

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system for developing and running the server
C / C++	Implementation of the asynchronous file processing server
GCC Compiler	Compiling the C source code
POSIX Threads (Pthreads)	Concurrent request processing and synchronization

8. Expected Outcome

The project is expected to produce an asynchronous file-processing server capable of handling multiple client requests concurrently. It will perform file read and write operations asynchronously, maintain request and completion queues, and safely synchronize shared data. The system will also measure request latency and server throughput to evaluate its performance

9. GitHub Repository

GitHub Repository Link: <https://github.com/marahrudaydeepak-ctrl/Asynchronous-File-Processing-Server>

Repository Created:

☐ Yes

Faculty Name and Signature

Faculty Name:

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____